Quick ADF Copy Activity Performance Tuning Runbook

Need deeper interpretation? See Quick Reference: Diagnosis-Driven Runbook (Word) | Quick Reference: Diagnosis-Driven Runbook (PDF) for explanations of waits, bottlenecks, and remediation choices. For complete methodology and remediation paths, use the Full ADF Copy Activity Performance Tuning Runbook (Word) | Full ADF Copy Activity Performance Tuning Runbook (PDF).

Goal

Find the limiting component with the fewest practical large-load executions. Do not run a full tuning matrix. Start with one representative baseline, use its evidence to choose one targeted test, and stop testing a setting as soon as it fails to produce a meaningful improvement.

Smallest practical test sequence

Run 1 - Baseline

Run the current/default configuration once using a representative workload.

Record only:

- Copy duration and throughput
- Rows copied/rejected
- DIUs actually used
- Parallel copies/connections used
- WRITELOG
- INSTANCE_LOG_RATE_GOVERNOR
- Log write latency / observed log MiB/s
- Live data/log IOPS when SQL MI appears pressured

Validate data correctness before tuning.

Diagnose the baseline

Baseline evidence	Obvious issue	Next test
ADF throughput is low and SQL MI log/I/O waits are low	ADF/source side may have unused capacity	Run one higher-DIU test
WRITELOG is high but INSTANCE_LOG_RATE_GOVERNOR is low	Commit/log-flush behavior may be limiting	Run one larger-batch test
INSTANCE_LOG_RATE_GOVERNOR is significant during sustained transfer	SQL MI is reaching its transaction-log write-rate limit	Do not test higher DIUs/writers first; test simplest sink/load design or MI capacity
One source file and SQL MI has headroom	File/source serialization may be limiting	Run one 4-file test
Multiple files exist but ADF uses little concurrency and SQL MI has headroom	Copy parallelism may be limiting	Run one parallel-copies test
Lock waits rise during concurrent loading	Writers are contending at SQL MI	Reduce concurrency; do not add writers
Data-file latency/IOPS rises while throughput plateaus	SQL MI data I/O is limiting	Investigate MI storage/capacity rather than ADF compute

Branch A - DIU test

Use only when the baseline does not already show SQL MI as the limiting component.

1. Baseline = current/Auto DIU.
2. Test one meaningful higher DIU value, normally the next practical supported level.
3. Compare throughput, duration, and SQL MI waits.

Decision:

- Improvement >=10% and SQL MI remains healthy: the change is useful. If additional scaling would matter, test one more DIU level.
- Improvement <10%: stop DIU testing. Additional ADF compute is not producing enough useful throughput.
- WRITELOG, log latency, or INSTANCE_LOG_RATE_GOVERNOR rises sharply while throughput improves <10%: stop increasing DIUs. ADF is pushing harder than SQL MI can use effectively.

Example:

```
8 DIUs:   100 MiB/s,  1 second INSTANCE_LOG_RATE_GOVERNOR
16 DIUs:  104 MiB/s, 75 seconds INSTANCE_LOG_RATE_GOVERNOR
```

Result: only 4% more throughput with sharply higher log-governor waiting. Keep 8 DIUs as the candidate and move to the SQL MI/log branch.

Branch B - Batch-size test under FULL recovery

SQL MI remains in FULL recovery. Do not design the tuning process around changing recovery models. Control transaction size, commit frequency, and writer pressure instead.

Use this branch when WRITELOG/commit behavior appears important but SQL MI is not already dominated by INSTANCE_LOG_RATE_GOVERNOR.

4. Baseline = current/default batch size.
5. Test one larger batch. For an unset/small baseline, 50,000 rows is a practical first comparison.
6. Compare duration/throughput, WRITELOG, active log usage, and log latency.

Decision:

- >=10% faster with comparable log behavior: keep the larger batch. Test one larger value only if additional improvement is worth pursuing.
- <10% faster: stop. Keep the smaller/default batch.
- Active log increases sharply, ACTIVE_TRANSACTION persists, rollback becomes much longer, or log latency worsens without >=10% throughput gain: reduce batch size.

If batch tuning does not remove a sustained log-rate bottleneck, stop spending runs on ADF settings and move to the SQL MI capacity path below.

Branch C - File-count test

Use when the baseline is a single large file and SQL MI has visible capacity headroom.

7. Baseline = one file.
8. Split the identical logical dataset into 4 files with split_delimited_file.py.
9. Use one Copy Activity against the multi-file source. Do not simultaneously introduce four independent SQL writers.
10. Compare end-to-end duration/throughput and SQL MI pressure.

Decision:

- 4 files are >=10% faster and SQL MI remains healthy: file-level parallelism helped. Stop here unless the remaining runtime justifies an 8-file test.
- Improvement <10%: do not test 8/16 files. Partitioning is not a useful first-order fix.
- ADF/source activity rises but end-to-end throughput plateaus and SQL MI waits rise: the sink is saturated. Stop increasing file count/concurrency.

Branch D - Parallel-copies test

Use when there are multiple source files/partitions, SQL MI has headroom, and ADF is not using enough concurrency.

11. Baseline = current/Auto parallel-copy behavior.
12. Test 2 parallel copies if baseline is effectively serial; otherwise test the next practical value.
13. If >=10% improvement occurs with stable SQL MI waits, test 4 only when the remaining runtime warrants it.

Stop immediately when the next level improves <10%, lock waits increase sharply, or log-governor waits rise while throughput plateaus.

SQL MI/log capacity escalation path

Use this path when the load is already constrained by SQL MI despite reasonable ADF settings.

Step 1 - Prove the sink is the constraint

Look for the combination:

- Additional DIUs/writers add <10% throughput
- INSTANCE_LOG_RATE_GOVERNOR or WRITELOG increases substantially
- Log write MiB/s/latency shows sustained pressure

Step 2 - Remove avoidable SQL work with one targeted comparison

If the target is indexed/constrained/triggered, compare the current load with an empty staging heap that has no unnecessary indexes, foreign keys, or triggers.

Measure total workflow time, not just ADF copy time, if data must later move into the final table.

Step 3 - If the simple heap still hits the log-rate limit

Stop tuning ADF. The next practical test is SQL MI capacity, not another DIU/batch/file-count experiment.

Temporarily scale the relevant MI capacity/configuration, then rerun the same workload and ADF settings once.

- Throughput improves >=10% and log-governor waiting falls materially: MI capacity was limiting the load. Evaluate the cost and whether temporary scale-up is operationally practical.
- Little improvement after MI scaling: investigate the next measured storage/network/database constraint rather than continuing blind ADF changes.

Confirmation - only after a likely winner exists

Do not repeat every exploratory test three times.

After one configuration clearly wins:

14. Run the original baseline one more time.
15. Run the candidate one more time.
16. If the candidate still improves >=10% with correct data and acceptable SQL MI health/cost, accept it for lab promotion.
17. If the repeated baseline itself changed >15%, the environment drifted; rerun only the baseline/candidate comparison, not the entire exploration.

Production acceptance

Lab results create a hypothesis. Production proves it.

18. Capture the current production run as P0 using the same core metrics.
19. Choose the single change supported by the lab diagnosis.
20. Apply that one change in the approved production test window.
21. Validate data correctness first.
22. Compare duration, throughput, SQL MI waits/latency, and cost with P0.
23. PASS: >=10% useful improvement with correct data and acceptable SQL MI/cost behavior.
24. FAIL: no useful improvement or operational regression; rollback.
25. INVESTIGATE: faster, but a new bottleneck/cost/risk appeared; do not promote yet.

Production context used while designing this runbook: approximately 134.04 GiB and 78,949,101 rows. For a workload of that size, the purpose of diagnosis-driven testing is to avoid weeks of exhaustive combinations. A baseline plus one or two targeted runs should eliminate most incorrect tuning paths before confirmation.

Writing and testing rule

Every instruction in this runbook should answer three questions plainly:

26. What did we measure?
27. What obvious problem does it indicate?

28. What is the smallest next test or action?